DEVOPS CAPSTONE · SESSION 21

# FleetPulse

Fleet maintenance tracking, shipped the DevOps way:
tests, containers, CI/CD with a security gate, Terraform on AWS, Helm on EKS, and Prometheus + Grafana.

**Parv Mehta**
github.com/Parv-0210/fleetpulse-devops-capstone

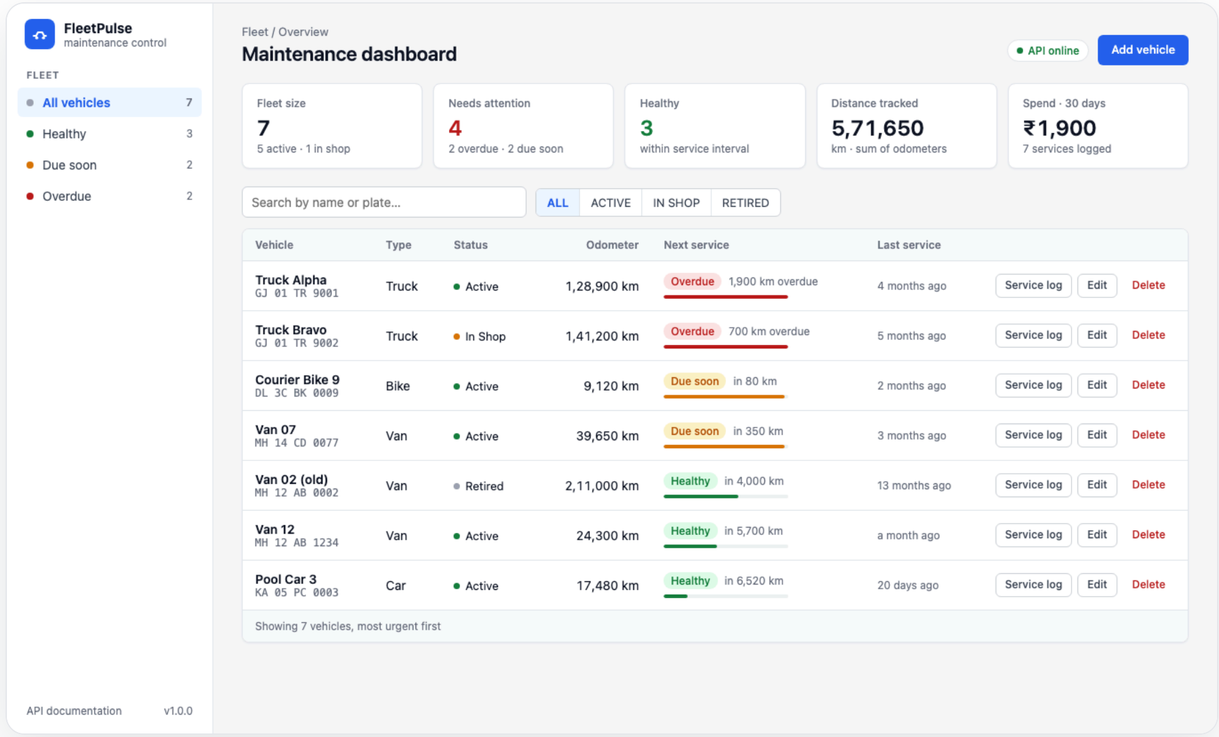
THE PROBLEM

# Which van is about to miss its service?

- Each vehicle has an **odometer**, a **service interval** and the **reading at its last service**
- From those three numbers FleetPulse computes the km left and a state:

| OK | Due soon | Overdue |
|---|---|---|
| plenty of distance left | ≤ 500 km to go | should already be serviced |

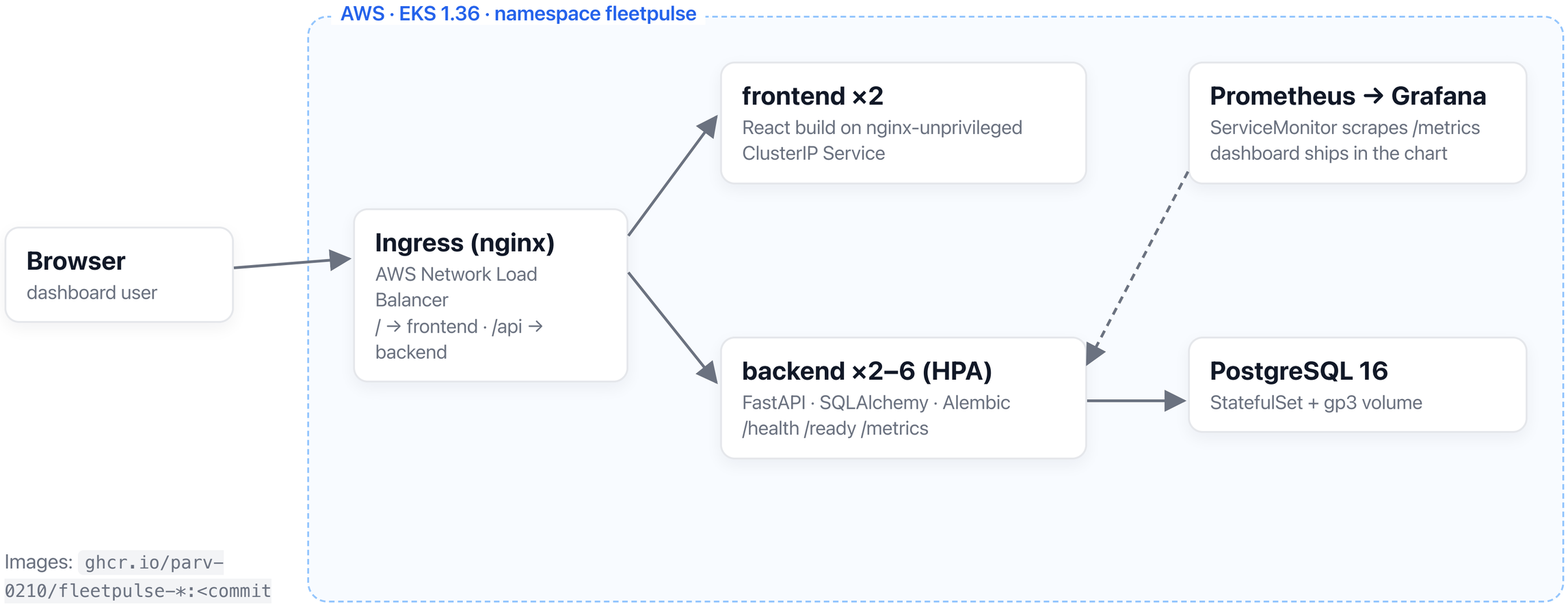
- Logging a service resets the counter; the dashboard shows the most urgent vehicles first and 30-day spend




The dashboard: React 19 + Vite, served by unprivileged nginx

ARCHITECTURE

# From a browser to PostgreSQL on EKS

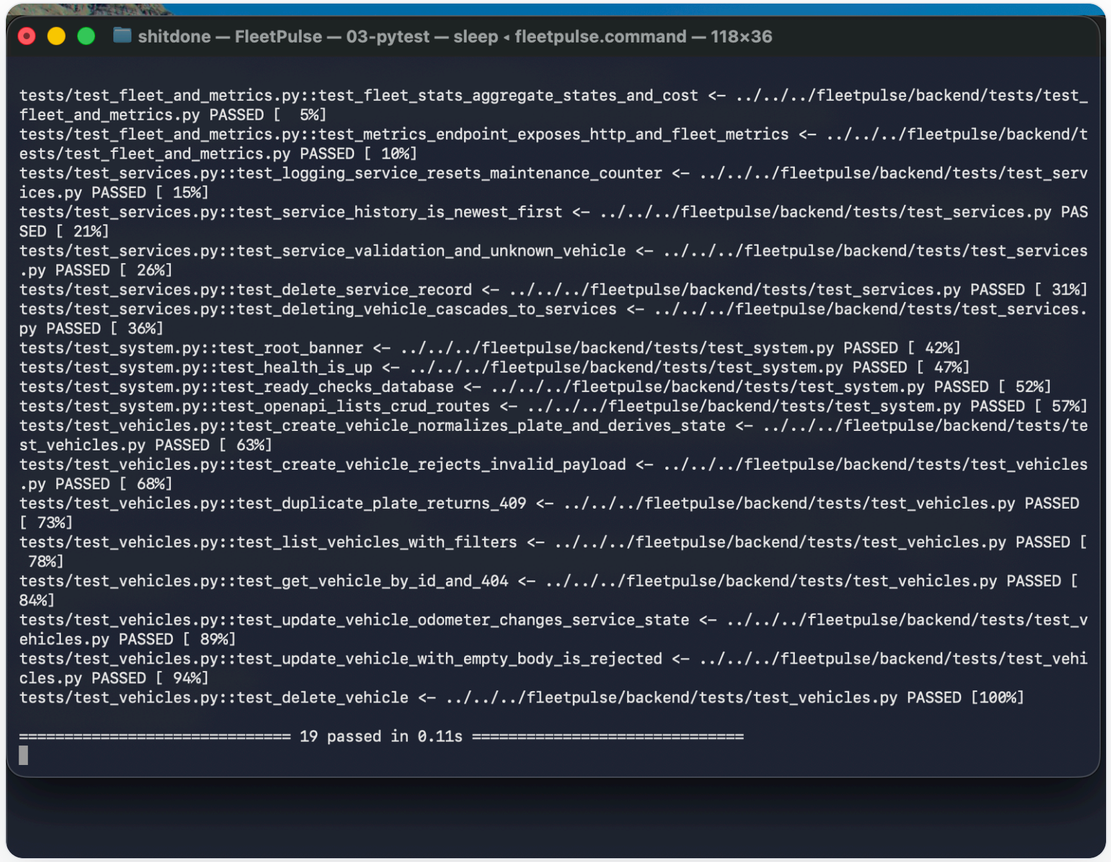



Images: `ghcr.io/parv-0210/fleetpulse-*:<commit sha>`

M1 · M2 — APPLICATION AND TESTS

# 12 REST endpoints, 19 tests

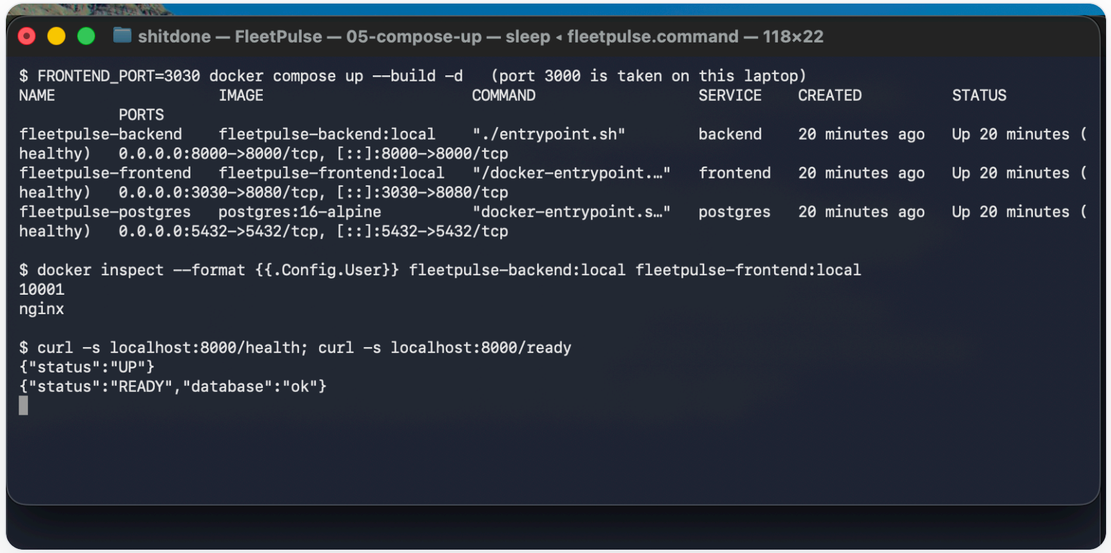
- **FastAPI** with full CRUD on `/api/vehicles`, service records, fleet stats
- **Two tables** built over two Alembic migrations: vehicles → service_records
- `/health` = process up · `/ready` = DB answers and tables exist
- **pytest**: 19 tests across system, vehicles, services and metrics
- Isolated test DB: SQLite in-memory locally, a real Postgres service in CI, via `get_db` override
- **ruff** lint + format check gate the build




`pytest -v` → 19 passed

# Small, non-root, multi-stage images

- **Backend**: two stages — build a venv, copy it into a slim runtime; runs as `USER 10001`; pip and setuptools removed
- **Frontend**: `node:22-alpine` build → `nginx-unprivileged:1.30-alpine-slim`, runs as uid 101
- **docker compose up --build**: Postgres, backend, frontend; backend waits for a healthy DB, runs migrations, seeds demo data
- Conventional commits, one per module; `.gitignore` keeps out .env, venvs, Terraform state




`docker compose up --build` — three services healthy

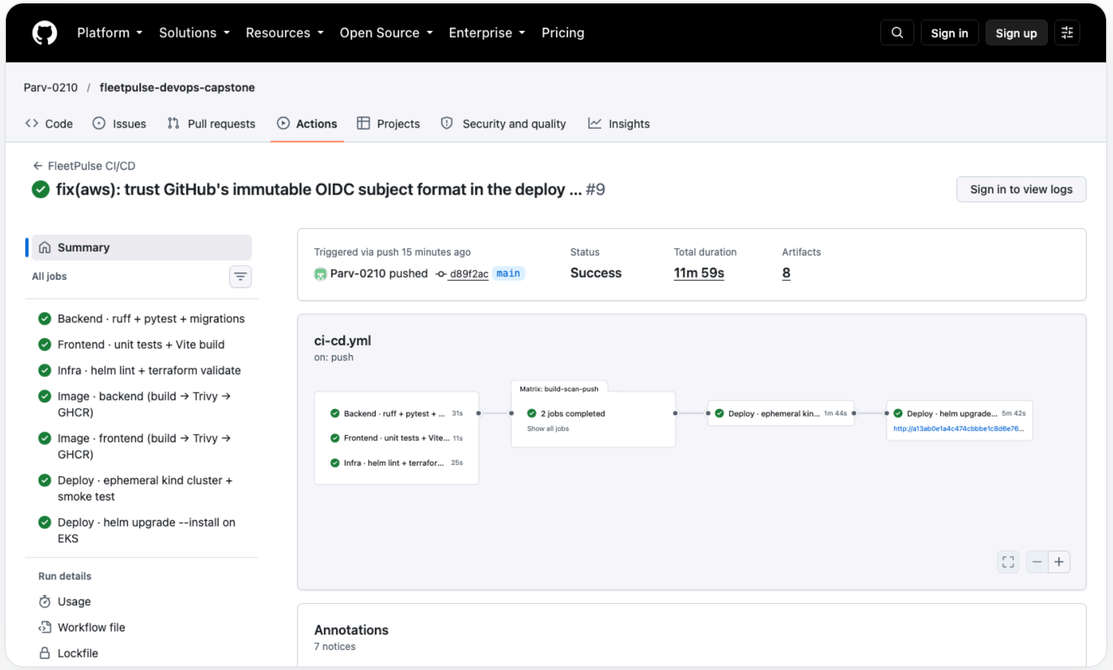
M5 — CI/CD WITH GITHUB ACTIONS

# Every push to main is tested, scanned and deployed

ruff + pytest + migrations | frontend tests + build | helm lint + tf validate → build amd64 + arm64 → Trivy gate →

push to GHCR :<sha> → kind smoke test → helm upgrade on EKS

- A failing test stops the image build — `needs:` chains the jobs
- Tags are the commit SHA, **never latest**: any pod traces back to its commit
- Throw-away **kind** cluster in CI: Helm install, `helm test`, curl the API
- EKS deploy signs in with **GitHub OIDC** — no AWS keys stored in GitHub



M6 — DEVSECOPS

# Trivy blocks any fixable HIGH or CRITICAL CVE

### Backend · first scan

**4**

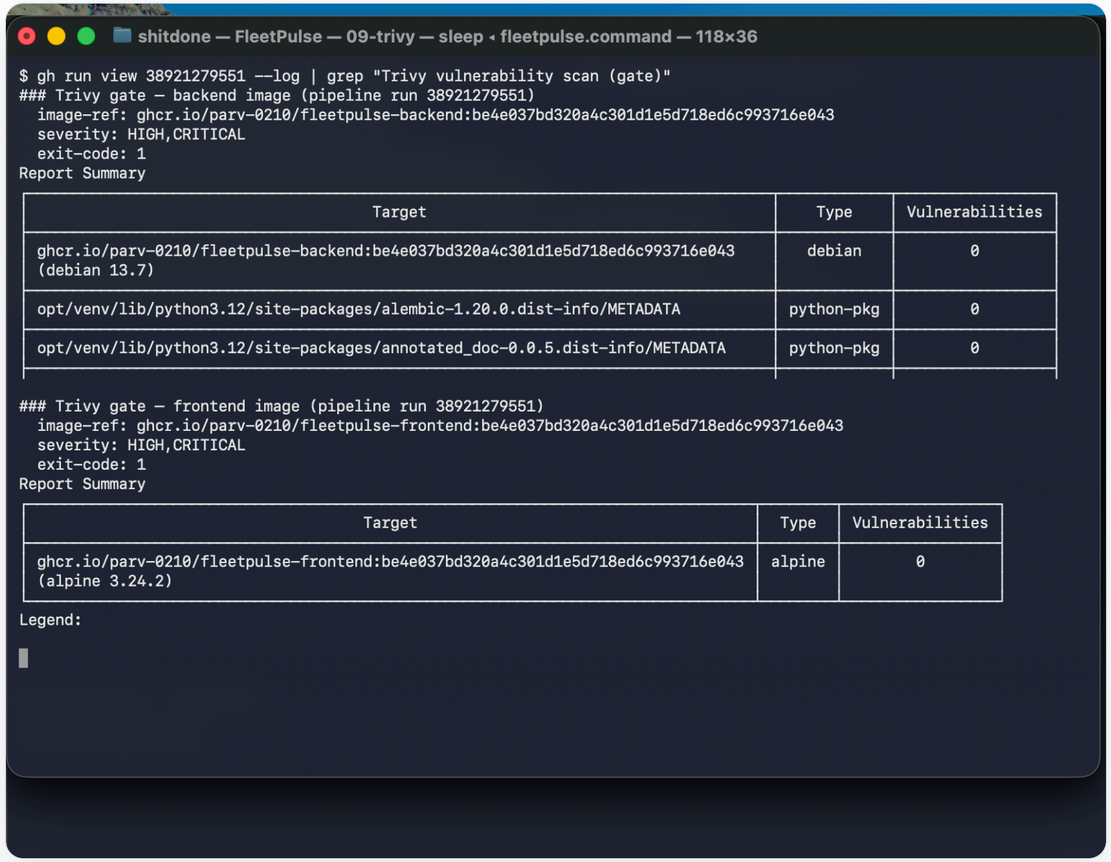
HIGH, all from pip / setuptools in the image

### Frontend · first scan

**41**

HIGH + CRITICAL from a stale Alpine nginx base

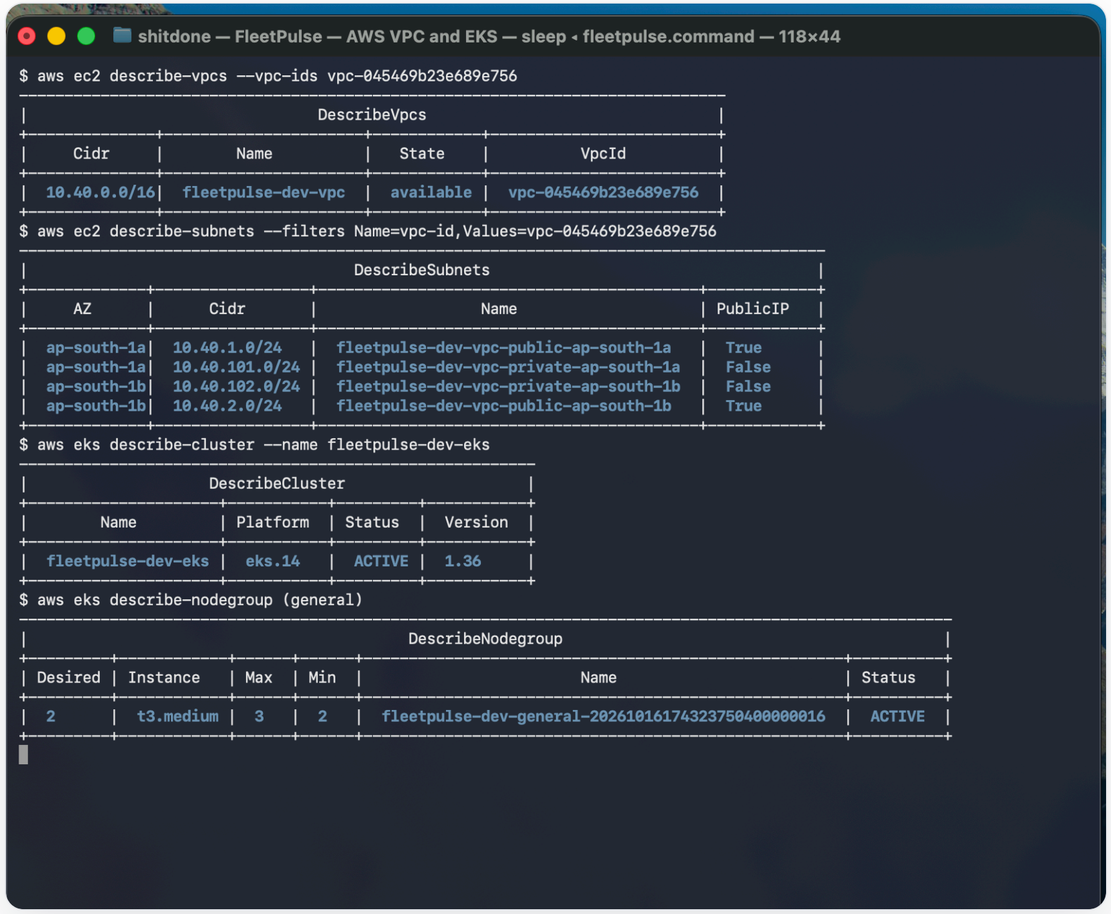
- Fix: strip pip from runtime, move to patched `nginx 1.30 alpine-slim`
- Now both images: **0 HIGH / 0 CRITICAL**
- Plus CodeQL, pip-audit, npm audit, secret scan of git history, IaC scan



# VPC + EKS in ap-south-1, 72 resources

- **VPC** `10.40.0.0/16`: 2 public + 2 private subnets across 2 AZs, NAT gateway, ELB tags
- **EKS 1.36** with a managed node group, `t3.medium × 2`
- **IAM role trusted by GitHub OIDC**, scoped to this repo's main branch
- Pinned modules: vpc 5.21 · eks 20.37 · iam 5.60
- `scripts/aws-up.sh` plan → apply → deploy; `aws-down.sh` tears it all down

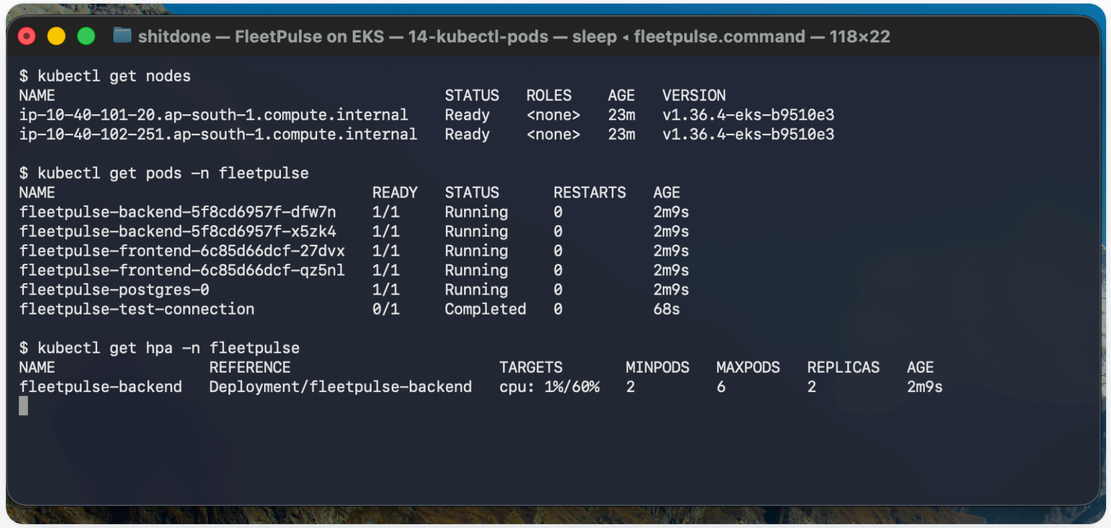
```
  # aws_iam_role.github_actions will be created
  # module.eks.aws_eks_addon.before_compute["vpc-cni"] will be created
  # module.eks.aws_eks_addon.this["aws-ebs-csi-driver"] will be created
  # module.eks.aws_eks_addon.this["coredns"] will be created
  # module.eks.aws_eks_addon.this["kube-proxy"] will be created
  # module.eks.aws_eks_addon.this["metrics-server"] will be created
  # module.eks.aws_eks_cluster.this[0] will be created
  # module.eks.module.eks_managed_node_group["general"].aws_eks_node_group.this[0] will be created
  # module.vpc.aws_internet_gateway.this[0] will be created
  # module.vpc.aws_nat_gateway.this[0] will be created
  # module.vpc.aws_subnet.private[0] will be created
  # module.vpc.aws_subnet.private[1] will be created
  # module.vpc.aws_subnet.public[0] will be created
  # module.vpc.aws_subnet.public[1] will be created
  # module.vpc.aws_vpc.this[0] will be created
  ...
Plan: 72 to add, 0 to change, 0 to destroy.
```




AWS CLI: VPC, subnets, cluster ACTIVE, node group ACTIVE

# One Helm chart, everything highly available

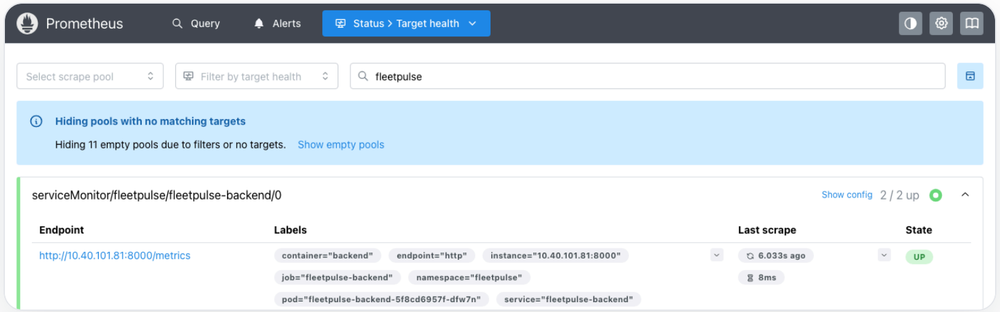
- Deployments with **2 replicas** each, **HPA** 2–6 on the backend, **PodDisruptionBudgets**
- ClusterIP Services; **Ingress** `/` → frontend, `/api` → backend on an AWS NLB
- Migrations in an init container behind a **Postgres advisory lock** — two pods never migrate at once
- Non-root, read-only root FS, all capabilities dropped; namespace enforces Pod Security *baseline*
- `helm test` checks the live release

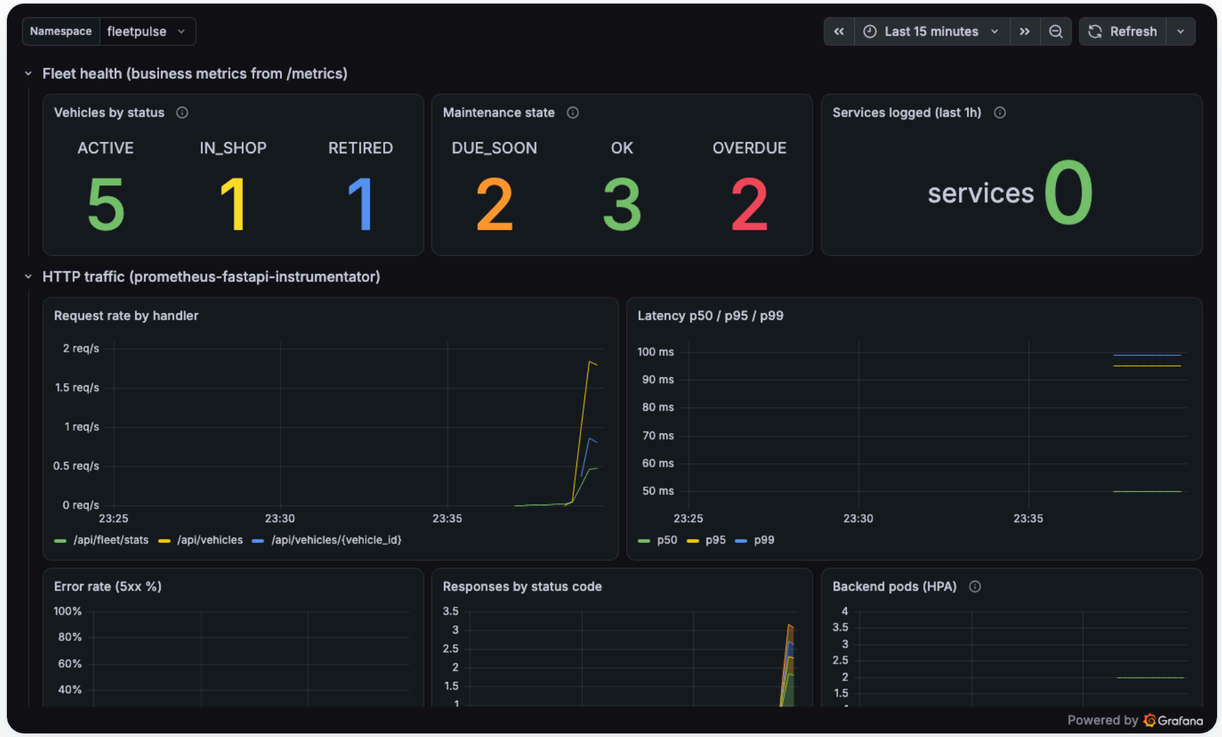





# Metrics about the fleet, not just about HTTP

- `/metrics` : request rate and latency from the instrumentator
- Plus business gauges: `fleetpulse_vehicles_by_service_state` — OK, due soon, overdue
- kube-prometheus-stack; a **ServiceMonitor** in the chart makes Prometheus scrape both backend pods
- The **Grafana dashboard is a ConfigMap in the chart** — deployed with the app, no manual import



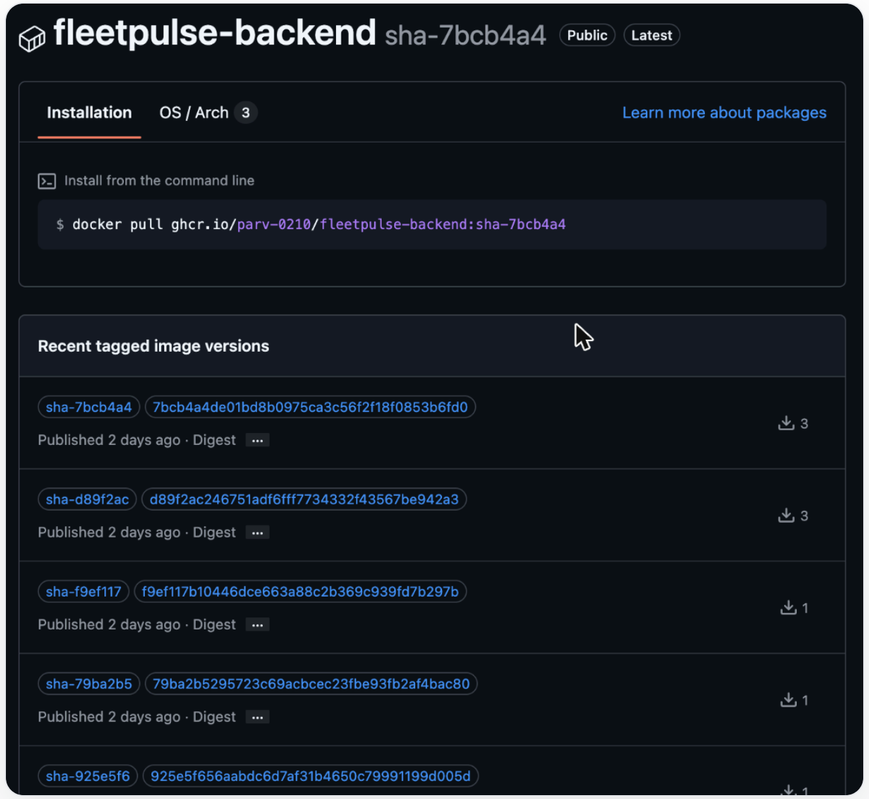




Grafana: fleet health next to API health

M10 — LIVE DELIVERY

# Commit → pipeline → new image → rollout

- `scripts/demo.sh` makes a visible UI change and commits it
- GitHub Actions runs tests, builds, scans and pushes `sha-<short>` images to GHCR
- The deploy job runs `helm upgrade --install` with those tags
- Pods roll over with zero downtime — the same SHA appears in git, in GHCR and in the running pods



GHCR: images tagged with the full and short commit SHA

LESSONS LEARNED

# What broke, and what I changed

### Security gate failed on day one

45 HIGH/CRITICAL findings came from base images, not my code. Removing pip and moving to a patched nginx fixed it.

### Two replicas, one migration

Pods started together and raced on Alembic. A Postgres advisory lock in an init container serialised them.

### OIDC subject format

The deploy role's trust policy had to accept GitHub's immutable subject format before the pipeline could assume it.

### Cost control

EKS costs ~0.25 USD/hour, so the cluster is created and destroyed by script; minikube covers free local testing.

THANK YOU

# FleetPulse

git push → tested → scanned → SHA-tagged image → Helm on EKS → watched in Grafana

**Parv Mehta**
github.com/Parv-0210/fleetpulse-devops-capstone